Multi-Source Transaction Normalization and Exploration

Individual Activity - KODIGO: Data Normalization and Structure

Student	Wilmer Salazar
Email	wilmerhenrysalazarmartinez@gmail.com
Repository	https://github.com/wkatir/genai-dev-critical-evaluation
Language	Python 3.11+ (interactive CLI built with *rich*)
Date	November 2, 2026

Summary

txnorm reads transactions from JSON files produced by four heterogeneous sources, detects the source of each record, normalizes every record to one canonical model, validates it, computes metrics and exposes the result through an interactive command-line interface with menus, filters, sorting, paging, a quarantine view and a per-record detail view that shows which rule was applied to each field. All transformation rules (status mapping, supported currencies and aliases, date formats, field mappings) live in a commented, validated configuration file: `config/rules.toml`.

How to run

```
git clone https://github.com/wkatir/genai-dev-critical-evaluation.git
cd genai-dev-critical-evaluation

# option A - uv
uv sync
uv run python -m txnorm

# option B - pip
python -m venv .venv && .venv\Scripts\activate
pip install -r requirements.txt && pip install -e .
python -m txnorm

python -m txnorm --summary          # metrics as JSON
python -m txnorm --export output    # normalized JSON/CSV, rejected records, metrics
pytest                              # 40 automated tests
```

Requirements coverage

Requirement	Implementation
Read transactions from JSON	`loader.py`: array or {"transactions": [...]}; several files at once; clear errors for missing files or broken JSON (line/column).
Detect type / source	`detection.py`: explicit marker field first, then structural signature; most specific match wins; ties rejected as AMBIGUOUS_SOURCE.

Requirement	Implementation
Normalize to a common format	`models.py` + `normalization.py`: one canonical Transaction (Decimal amount, ISO currency, 5 statuses, UTC datetime).
Amount conversion	Decimal (never float); per-source decimal separator (1,234.56 vs 1.234,56) with strict grouping; minor units (cents) converted; rounding half-even, flagged.
Currency normalization	ISO codes case-insensitive, aliases ($, US$, EUR sign, euro...), per-source override ($ = MXN for the Mexican POS).
Status mapping	Global aliases per canonical status plus per-source codes (wallet: 1, 0, -1, 2, 3).
Date formats	iso8601, epoch seconds/ms or strptime patterns per source; naive times take the source timezone; all stored in UTC.
Invalid / incomplete records	`validation.py`: 13 issue codes (11 errors, 2 warnings). Invalid records are separated into a quarantine with all their issues and the raw record, never silently dropped.
Metrics	`metrics.py`: total processed, valid vs invalid, count by status, totals by currency (all and completed only), by source, by issue code.
Interactive interface	`interface/cli.py`: dashboard, browse with filters/sort/paging, detail, quarantine, issue report, rules view, export, load files.
Rules / configuration file	`config/rules.toml`: status mapping, supported currencies, aliases, date formats, source mappings; validated at startup.
Test data	`data/valid_transactions.json` (20 valid) and `data/edge_cases.json` (23 inconsistent records, each with its expected outcome).
Technical note	`TECHNICAL_NOTE.md` (design decisions and use of AI) - also reproduced in this document.

Results on the sample data

Processed	Valid	Invalid (quarantined)	Valid with warnings	Sources	Currencies
43	24 (56%)	19	2	4	USD, EUR, GBP, MXN, COP, JPY

Every issue code is exercised by the edge-case file: MISSING_FIELD, INVALID_AMOUNT, NON_POSITIVE_AMOUNT, UNSUPPORTED_CURRENCY, UNKNOWN_STATUS, INVALID_DATE, DATE_OUT_OF_RANGE, DUPLICATE_ID, INVALID_ID, UNKNOWN_SOURCE, MALFORMED_RECORD, plus the AMOUNT_ROUNDED and TIMEZONE_ASSUMED warnings. AMBIGUOUS_SOURCE is covered by a unit test with dedicated rules.

Project structure

Path	Responsibility
config/rules.toml	Normalization rules: status mapping, currencies, aliases, date formats, sources
src/txnorm/models.py, config.py	Canonical model and issue codes; rules loading and validation
loader.py, detection.py, pipeline.py	JSON reading; source detection; orchestration of the process
normalization.py, validation.py	Field conversion rules; business rules and duplicate policy
metrics.py, query.py, export.py	Aggregates; filtering and sorting; JSON and CSV output
interface/cli.py	Interactive interface (renders only, no business logic)
data/, tests/	Test data with valid and inconsistent records; automated tests

Technical note

Design decisions

- **Two-level model.** *Transaction* is the strict canonical schema (typed, immutable, only built when a record has no errors). *RecordResult* is an audit envelope kept for every input record: detected source, partial normalized fields, a per-field trace of the rule applied, issues and the raw data.
- **Rules as data.** Mappings, currencies, aliases and date formats live in a commented TOML file read by the standard library and validated at startup, so a bad rule fails loudly instead of corrupting data.
- **Money is Decimal**, quantized to each currency's decimals. Totals are only reported per currency: without an exchange-rate source any cross-currency total would be invented.
- **All timestamps in UTC**, with source-specific local timezones for values without an offset.
- **Separate, don't discard.** Invalid records are quarantined with all their issues (normalization does not stop at the first error) and excluded from totals; warnings flag a record but keep it. The first *valid* occurrence of an id wins over duplicates, so a corrected resend is accepted.
- **Layering.** loader, detection, normalization, validation, metrics and query are independent of the interface; the CLI only renders, so another interface could reuse the same core.

Use of AI

Claude Code (Anthropic) was used as a pair programmer: it proposed the module layout, drafted the parsers, the CLI and the test data, and ran the test suite after every change. Suggestions reviewed and adjusted:

Initial suggestion	Adjustment	Reason
Parse amounts as float	Decimal, quantized per currency	0.1 + 0.2 != 0.3; money must be exact
Map $ to USD globally	Per-source alias override ($ = MXN for the POS)	$ is ambiguous across countries
Accept 1234.56 in a comma-decimal source	Strict grouping, reject as ambiguous	Guessing the separator silently corrupts amounts
Refunds as negative amounts	Amount always > 0; direction comes from status	Avoids double negation and ambiguous totals
Stop at the first error per record	Collect every issue of the record	Users fix data faster when they see all problems
One grand total of all amounts	Totals per currency, also completed only	Adding USD and COP is meaningless; failed payments are not income
Any repeated id is a duplicate	Only a valid record claims an id	A corrected resend must be accepted

Verification. 40 automated tests (parsers, business rules, metrics, filters, configuration errors and scripted CLI sessions) plus a data-driven test that checks the expected outcome written in each record of the edge-case file. Issues found during manual review were fixed: the `[n]ext` prompt was swallowed by terminal markup (escaped), and an invalid filter value aborted the remaining filters (each field is now re-asked until valid).